

52.2. La fonction `silence()` — faire taire une voix

`#include <silence>` : `silence(voix)` coupe la voix 1, 2 ou 4.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `silence(voix)` coupe la voix tout de suite : une note sans fin, un air, un bruit.

La ligne à écrire : `#include <silence>`.

Son argument : la voix : 1, 2, ou 4 pour le bruit.

Ce qu'elle coûte : une vingtaine d'octets.

Essaie : coupe la note quand on appuie sur B.

On retrouve silence dans 2 leçons

[53](#), [54](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **581 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      61 o ← la console, à l'allumage – la mise en route de la console
main()         86 o ← démarrage – ta fonction
VBlank         8 o  ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        144 o ← CopierTuiles – 9 lettres sur 43
Notes         120 o ← note() – la table des notes de musique
"UN DO PUIS RIEN" 15 o ← texte() – un texte gravé
```

Lettres gravées : DEINOPRSU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert

Fonctions du langage appelées :

```
note()    x 1    55 o sur place, dans main() + Notes (120 o)
texte()   x 1    11 o sur place, dans main() + EcrireTexte (10 o)
silence() x 1     6 o sur place, dans main()
image()   x 1     3 o sur place, dans main() + AttendreImage (28 o)
images()  x 1     2 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

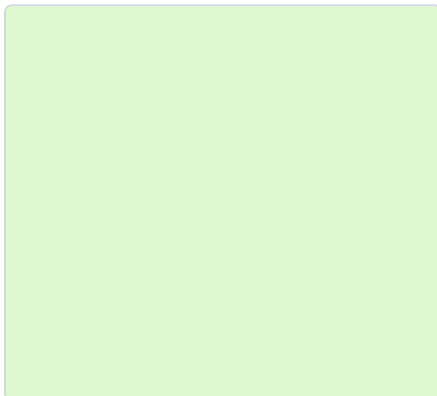
```
// ---- #include <silence> : faire taire une voix ----  
// Une note sans fin, qui se tait au bout d'une seconde.  
  
#include <note>  
#include <silence>    // silence() : couper une voix  
#include <texte>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

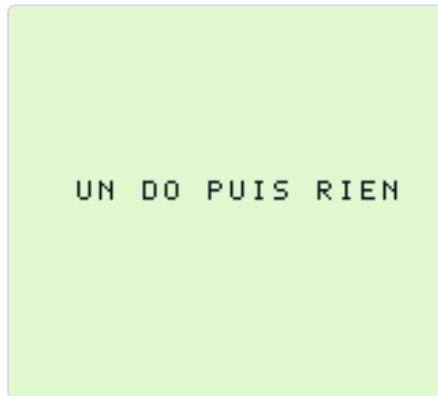
```
int main() {  
    texte(3, 8, "UN DO PUIS RIEN");  
    note(1, D04, 0, 12);    // D04, sans fin  
  
    while (true) {  
        image();  
        if (images() == 60) {    // une seconde après l'allumage  
            silence(1);        // la voix 1 se tait  
        }  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un DO qui sonne une seconde, puis le silence.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la note joue au départ
- ✓ une seconde plus tard, la voix 1 se tait

